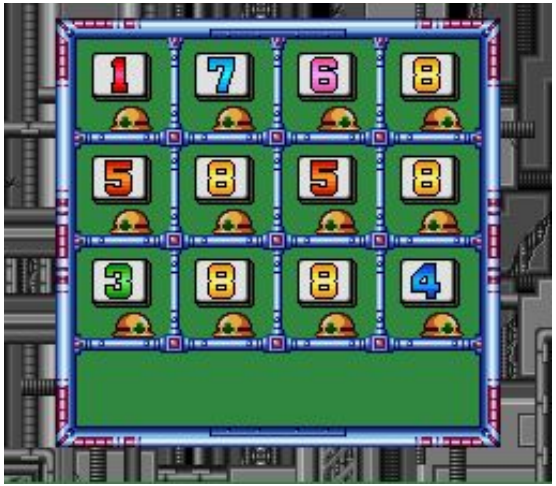
Programação 2

Aula 11 - Orientação a Objetos: encapsulamento

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Recapitulando

É pra começar do zero, toda vez?

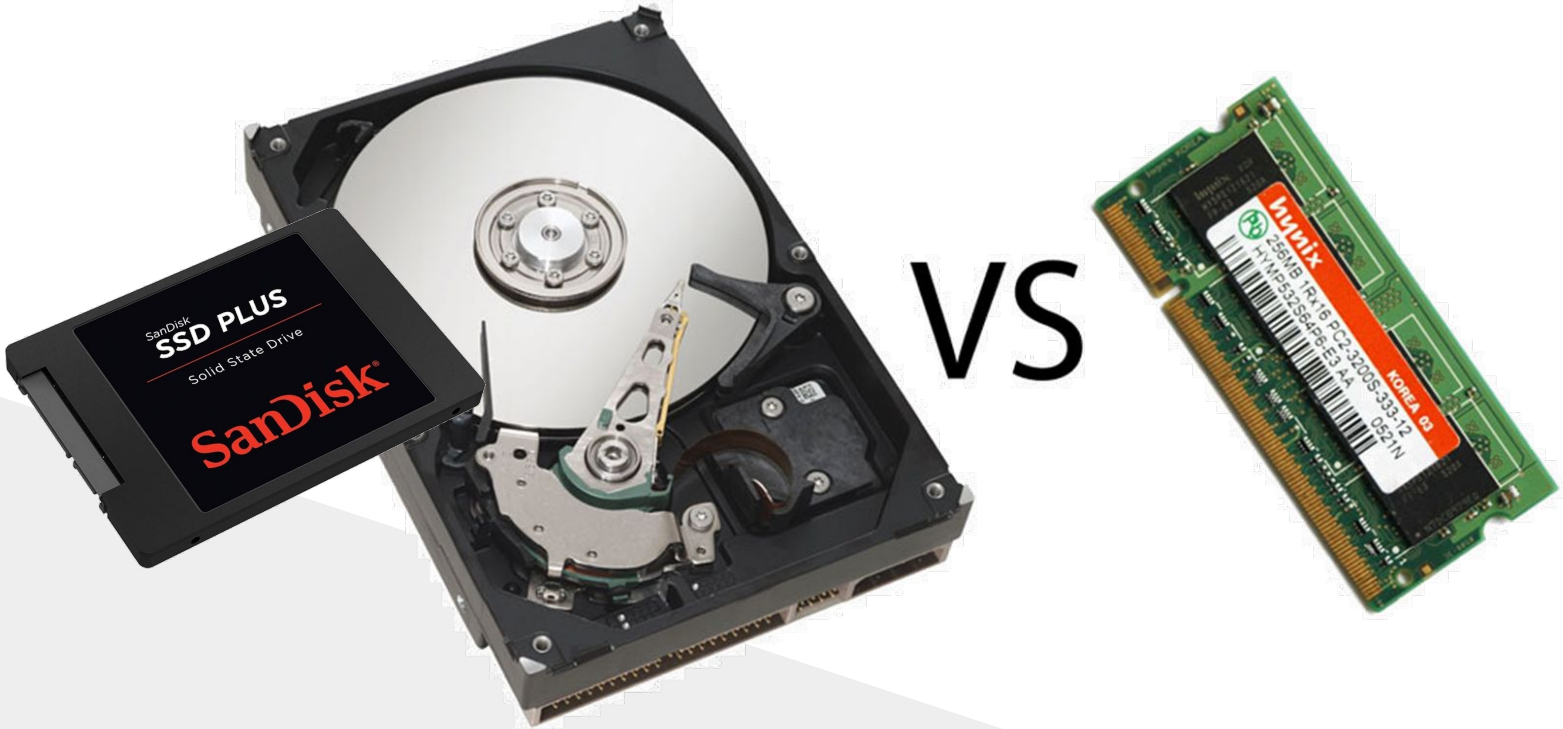


Os estados de um programa (situação atual dos dados) varia

Ao fechar o programa a memória RAM é liberada

Tem como não perder o progresso?

HD vs RAM



LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Leitura e Escrita de Arquivos para Persistência

Arquivos não desaparecem após fechar o programa

Todo o conteúdo da aplicação pode ser depositado em arquivos

- É uma boa prática inclusive, não incluir conteúdo no código

Neste contexto, conteúdo se refere a tudo que não define a lógica do programa

- Nomes de personagem, valores de energia, textos dos capítulos, etc
- Imagens
- Vídeos
- Modelos 3D
- Etc

Criando Arquivos em Java

Classe File

```
try {
  File myObj = new File("filename.txt");
  if (myObj.createNewFile()) {
    System.out.println("File created: " + myObj.getName());
  } else {
    System.out.println("File already exists.");
  }
} catch (IOException e) {
  System.out.println("An error occurred.");
  e.printStackTrace();
}
```

Java Create and Write To Files (w3schools.com)

Escrevendo em Arquivos Java com FileWriter

```java
try {
  FileWriter myWriter = new FileWriter("filename.txt");
  myWriter.write("Files in Java might be tricky, but it is fun enough!");
  myWriter.close();
  System.out.println("Successfully wrote to the file.");
} catch (IOException e) {
  System.out.println("An error occurred.");
  e.printStackTrace();
}
```

[Java Create and Write To Files (w3schools.com)]

Lendo Arquivos em Java com Scanner

File + Scanner

```java
import java.util.Scanner;
import java.io.File;
import java.io.FileNotFoundException;

public class Test
{
    Run | Debug
    public static void main(String[] args)
    {
        File arquivo = new File("rsc/Arquivo.txt");

        try {
            Scanner escaneador = new Scanner(arquivo, "UTF-8");

            int i = 0;
            while (escaneador.hasNextLine()) {
                i++;
                String leitura = escaneador.nextLine();
                System.out.println("Linha "+ i + ": " + leitura);
            }
            escaneador.close();
        }
        catch (FileNotFoundException e) {
            e.printStackTrace();
        }
```

Arquivo

xspace\Java\Projects\First Project\FirstProject\rsc\Arquivo.txt - Sublime Text (UNREGISTERED)

```
1  Esta é a primeira linha
2  Esta é a segunda...
3  .
4  .
5  Esta é a quinta e última linha
```

Console

d:\Workspace\Java\Projects\First Project\FirstProject - VS Code Console

```
Linha 1: Esta é a primeira linha
Linha 2: Esta é a segunda...
Linha 3: .
Linha 4: .
Linha 5: Esta é a quinta e última linha
Press any key to continue . . .
```

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Escrevendo em Arquivos Java com FileReader

FileReader

```java
import java.util.Scanner;
import java.io.File;
import java.io.FileNotFoundException;

public class Test
{
    Run | Debug
    public static void main(String[] args)
    {
        File arquivo = new File("rsc/Arquivo.txt");

        try {
            Scanner escaneador = new Scanner(arquivo, "UTF-8");

            int i = 0;
            while (escaneador.hasNextLine()) {
                i++;
                String leitura = escaneador.nextLine();
                System.out.println("Linha "+ i + ": " + leitura);
            }
            escaneador.close();
        }
        catch (FileNotFoundException e) {
            e.printStackTrace();
        }
```

Arquivo

xspace\Java\Projects\First Project\FirstProject\rsc\Arquivo.txt - Sublime Text (UNREGISTERED)

```
1  Esta é a primeira linha
2  Esta é a segunda...
3  .
4  .
5  Esta é a quinta e última linha
```

Console

d:\Workspace\Java\Projects\First Project\FirstProject - VS Code Console

```
Linha 1: Esta é a primeira linha
Linha 2: Esta é a segunda...
Linha 3: .
Linha 4: .
Linha 5: Esta é a quinta e última linha
Press any key to continue . . .
```

Dicionários (HashMap)

Arrays são acessados pelo índice, mas e se quiséssemos acessar cada item pelo nome do personagem. Ou seja, usar o nome como a "**chave**" de acesso.

```
Map<String, Personagem> personagens = new HashMap<String, Personagem>();
personagens.put("Julia", new Personagem("Julia"));
personagens.put("Julia", new Personagem("Julia"));
personagens.put("André", new Personagem("André"));
personagens.put("Alfa", new Personagem("Alfa"));

personagens.get("Julia");

for (Map.Entry<String,Personagem> personagem : personagens.entrySet()) {
    System.out.println("Chave: " + personagem.getKey());
}
```

Conceito de Hoje

Orientação a Objetos: encapsulamento

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Introdução

Problema

Motivação

História e Pessoas

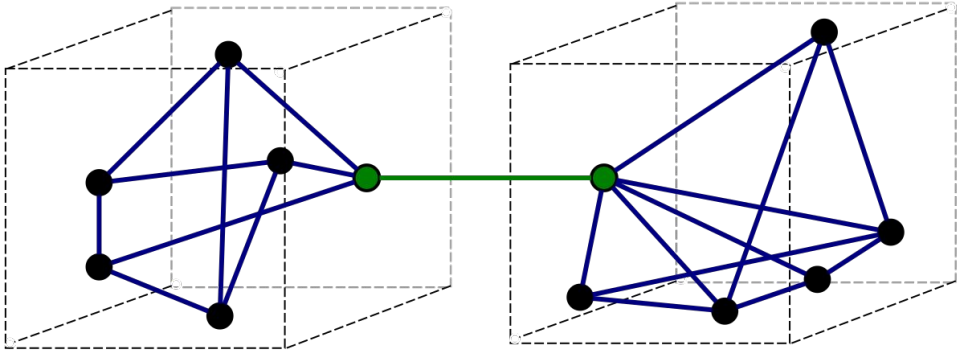
Coesão e Acoplamento

Coesão

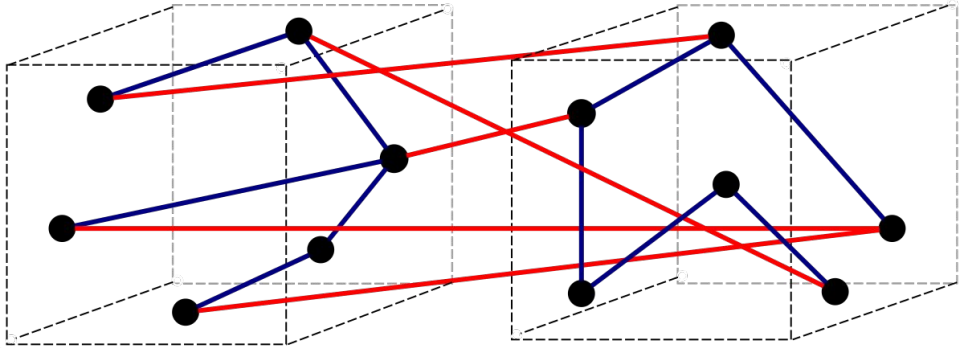
Se refere ao relacionamento que os membros de um módulo possuem. Códigos coesos são aqueles de relação forte, onde seus **membros estão intimamente ligados e estão ali por um objetivo comum**. Membros que não são absolutamente necessários para aquele módulo não devem estar presentes em códigos coesos.

Acoplamento

Se refere ao relacionamento entre os módulos. Indica **quanto um módulo depende de outro** para funcionar. Códigos desacoplados são aqueles de relação fraca, e não dependem de outros para fazer sua funcionalidade básica. É sempre desejável um baixo nível de acoplamento.



a) Good (loose coupling, high cohesion)



b) Bad (high coupling, low cohesion)

[java - O que são os conceitos de coesão e acoplamento? - Stack Overflow em Português]

Apresentação

Definições

Exemplos Ilustrativos

1.5.7 Encapsulamento e ocultamento de informações

Classes (e seus objetos) **encapsulam**, isto é, contêm seus atributos e métodos. Os atributos e métodos de uma classe (e de seu objeto) estão intimamente relacionados. Os objetos podem se comunicar entre si, mas eles em geral não sabem como outros objetos são implementados — os detalhes de implementação permanecem *ocultos* dentro dos próprios objetos. Esse **ocultamento de informações**, como veremos, é crucial à boa engenharia de software.

Modificadores de Acesso: **public** vs **private**

public

indica que aquele elemento (ex.: classe, atributo, método...)
é acessível dentro e fora daquela classe

private

indica que aquele elemento (ex.: classe, atributo, método...)
é acessível somente dentro daquela classe

é uma forma de proteger/limitar o acesso

uma boa prática de programação é sempre usar *private* como
padrão e somente tornar algo *public* quando necessário

Métodos de Acesso e Atualização: **get** , **set**

get

É comum se usar um método *get* para acessar uma variável. O uso deste tipo de recurso possibilita garantir que aquela variável não será alterada de forma indevida.

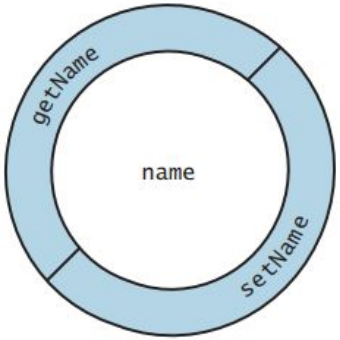
set

O método *set* é comumente utilizado na definição das classes para garantir que o caminho de alteração das variáveis da instância sempre ocorram por um único fluxo.

Métodos de Acesso e Atualização: **get** , **set**

Visualização conceitual de um objeto `Account` com dados encapsulados

Você pode pensar em um objeto `Account` como o mostrado na Figura 3.4. A variável de instância `private` chamada `name` permanece *oculta no* objeto (representado pelo círculo interno contendo `name`) e *protegida por uma camada externa* de métodos `public` (representados pelo círculo externo contendo `getName` e `setName`). Qualquer código do cliente que precisa interagir com o objeto `Account` *só* pode fazer isso chamando os métodos `public` da camada externa protetora.



Métodos de Acesso e Atualização: **get** , **set**

Motivações

1. Possibilidade de você criar rotinas de validação na atribuição (seu Setter seria um procedimento que pode sinalizar com exceções um valor inválido). Olhe este exemplo (que verifica se o número atribuído é zero e lança uma exceção caso seja).

```
public void setNumero(int numero){
    if (numero ==0)
        throw new RuntimeException("Numero invalido");
    else
        this.numero = numero;
}
```

2. Possibilidade de você criar um atributo que é acessível, mas não modificável (read-only), ou seja, você tem um atributo que só tem SET e não tem SET (a modificação do valor do atributo ficaria a cargo de um método de cálculo, por exemplo)

Código Produzido em Sala

```java
public static void main(String[] args) throws Exception {
    Pessoa matheus = new Pessoa(
        cpf: "076.664.214-44",
        nome: "Matheus Gabriel Vila Nova de Freitas Oliveira",
        idade: 20,
        senhaEmail: "senha123");

    Pessoa cleyson = new Pessoa(
        cpf: "126.454.268-34",
        nome: "Cleyson Silva de Oliveira",
        idade: 20,
        senhaEmail: "91438913");

    //System.out.println(matheus.senhaCartao);
    //cleyson.nome = "José Antônio";
    //matheus.senhaEmail = "123456";
    System.out.println(cleyson.getNome());
    matheus.setNota(nota: 9.5f);
    System.out.println(matheus.getNota());
    System.out.println(cleyson.getCpf());
}
```

```java
public class Pessoa {
    private String cpf;
    private String nome;
    private String senhaEmail;
    private float nota;

    public Pessoa(String cpf, String nome, int idade, String senhaEmail) {
        this.cpf = cpf;
        this.nome = nome;
        this.senhaEmail = senhaEmail;
        this.nota = 0.f;
    }

    public String getNome() {
        return this.nome;
    }

    public void setNota(float nota) {
        this.nota = nota;

        if(nota < 0.0f) {
            this.nota = 0.0f;
        }
        else if(nota > 10.0f) {
            nota = 10.0f;
        }
    }

    public float getNota() {
        return this.nota;
    }

    public String getCpf() {
        String meio = this.cpf.substring(beginIndex: 4, endIndex: 10);
        return "***" + meio + "-**";
    }
}
```

Exercício

Mini-Projeto 7

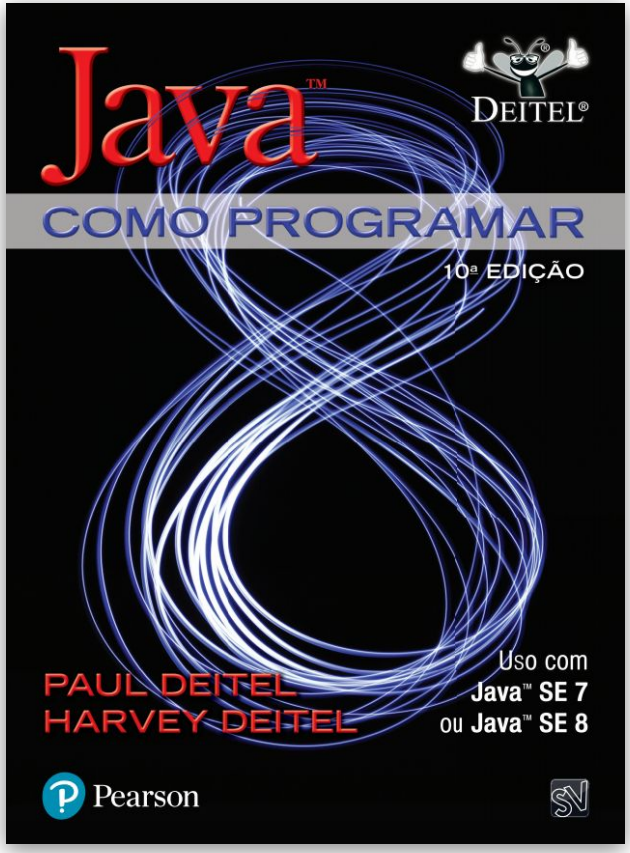
Todos os atributos de todas as classes devem ter o modificador de acesso **private**

Todos os métodos que não são necessários para outras classes devem ter o modificador de acesso **private**, caso contrário escrever explicitamente o modificador de acesso **public**

Criar métodos **get** somente para os atributos que precisam ser acessados por outras classes

Criar métodos **set** somente para os atributos que precisam ser modificados por outras classes

Métodos set devem fazer checagens para impedir modificações inválidas (ex.: energia do personagem menor que 0)

Referências

Java - Como Programar (10a Edição)